

TECHNICAL REQUIREMENTS DOCUMENT

Industrial Knowledge Intelligence Platform

System Responsibilities, Data Ownership & Component Interactions

Version 0.1 · Internal Working Draft

Prepared for the engineering team ahead of implementation planning

Prepared by Musa

July 2026

Table of Contents

Table of Contents	2
1. Purpose	3
2. Platform Overview	3
3. Identity System	4
4. Knowledge Ingestion System	5
5. Knowledge Platform	7
6. Search System	8
7. AI Intelligence Layer	9
8. RAG Pipeline	9
9. Knowledge Relationships	11
10. AI Agents	11
11. Storage Layer	12
12. Cache Layer	13
13. Background Processing	13
14. API Layer	15
15. User Platform	15
16. Event System	16
17. Security Requirements	16
18. Observability	17
19. Component Dependency Flow	18
—. My Biggest Recommendation Before Writing Any Code	19

1. Purpose

This document defines the technical components required to build the Industrial Knowledge Intelligence Platform.

It focuses on system responsibilities, data ownership, and component interactions, rather than implementation details.

2. Platform Overview

The platform has one primary responsibility:

Convert scattered industrial documents into structured organizational knowledge that AI can search, reason over, and present to users.

Everything else supports this goal.

The platform consists of six major systems:

#	System
1	Identity System
2	Knowledge Ingestion System
3	Knowledge Platform
4	AI Intelligence Layer
5	User Platform
6	Infrastructure Platform

3. Identity System

Manage organizations, users, authentication, permissions, and security.

Without this layer, every document would be public.

RESPONSIBILITIES	● Login ● Registration ● Organizations ● Team Members ● Roles ● Permissions ● Sessions ● API Authentication
INPUTS	● Login request ● User invitation ● Organization creation
OUTPUTS	● JWT / Session ● User profile ● Organization context
OWNS	● Users ● Organizations ● Roles ● Permissions

4. Knowledge Ingestion System

This is the heart of the product. Everything begins here.

Purpose

Transform raw documents into structured knowledge.

Responsibilities

- Accept uploaded files.
- Store originals.
- Extract text.
- Extract metadata.
- Split documents.
- Generate embeddings.
- Extract entities.
- Extract relationships.
- Send data to storage.

Supported Documents

- PDF
- DOCX
- PPT
- Excel
- CSV
- Images
- Scanned PDFs
- Emails
- P&ID
- Engineering Drawings

Internal Pipeline

Every uploaded document passes through this pipeline:



5. Knowledge Platform

The Knowledge Platform stores the intelligence created during ingestion.

It is not responsible for AI. It only stores and organizes knowledge.

RESPONSIBILITIES	● Store document metadata. ● Store document chunks. ● Store embeddings. ● Store entities. ● Store relationships. ● Maintain references. ● Maintain document versions.
OWNS	● Documents ● Chunks ● Metadata ● Equipment ● Relationships

6. Search System

Purpose: allow users and AI to retrieve relevant knowledge.

Supports

- Keyword Search
- Semantic Search
- Hybrid Search
- Equipment Search
- Document Search
- Filtered Search

Rule

Search should never access raw PDFs directly. It always searches processed knowledge.

7. AI Intelligence Layer

This layer never owns data. It consumes data. Think of it as a reasoning engine.

Responsibilities

- Understand user intent.
- Retrieve knowledge.
- Reason over context.
- Generate responses.
- Provide citations.
- Ask clarifying questions.
- Maintain conversations.

Rule

It never directly searches storage. It always goes through the Search System.

8. RAG Pipeline

This is where many people get confused. RAG is not the product. It is one component.

When a user asks: "Why did Pump P101 fail?" — the platform performs:

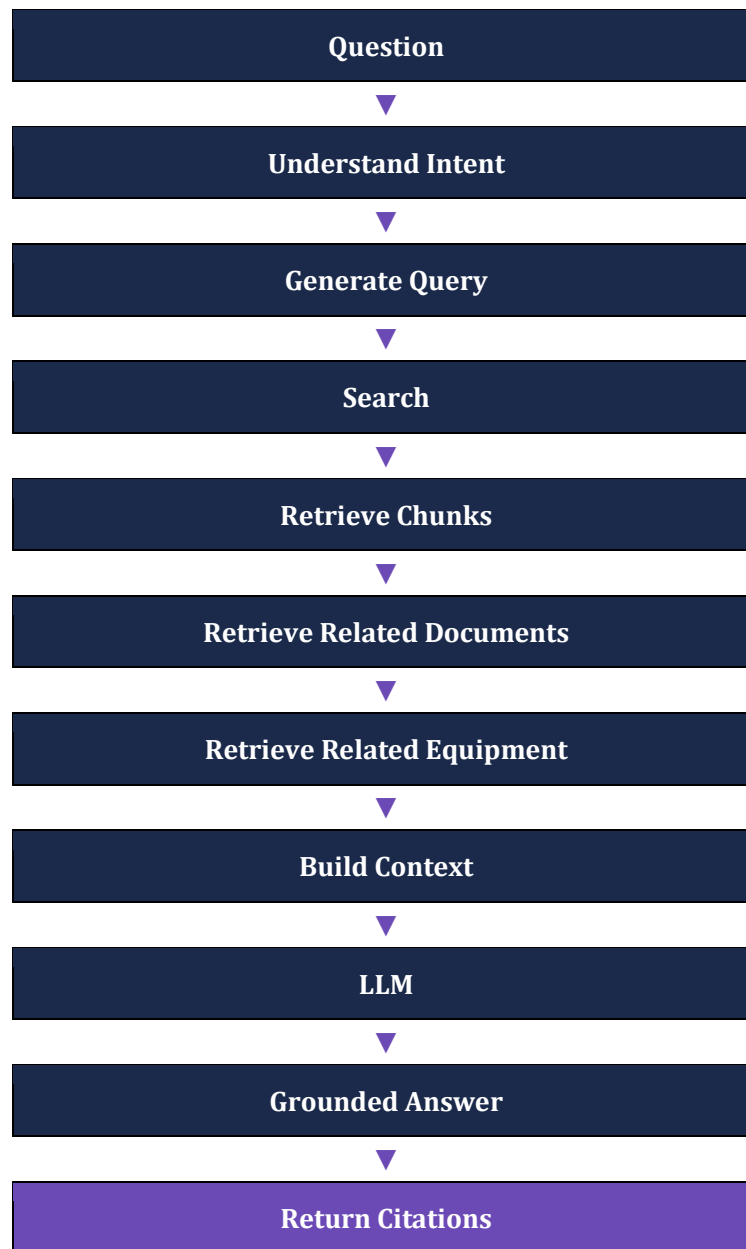


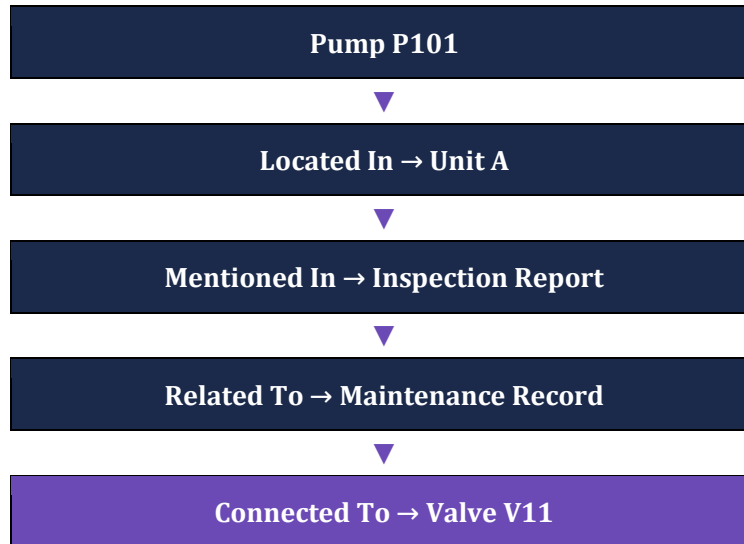
Figure — Retrieval-Augmented Generation flow for a single user question

Note

The LLM never reads the database. It only reads prepared context.

9. Knowledge Relationships

Relationships are created during ingestion. Example:



These relationships help retrieval. They also power future agents.

10. AI Agents

Agents are not part of document ingestion. They are consumers of the knowledge platform.

Example Agents

- Maintenance Agent
- Compliance Agent
- Incident Investigation Agent
- Lessons Learned Agent
- Knowledge Assistant

Every agent retrieves knowledge using the same platform.

11. Storage Layer

The platform stores several different types of information. Each type has different requirements.

Data Type	Purpose	Examples
Original Documents	Keep uploaded files.	PDF, DOCX, Images
Metadata	Information about documents.	Title, Author, Upload Date, Department, Equipment, Version
Document Chunks	Small searchable sections.	—
Embeddings	Semantic search.	—
Relationships	Connect knowledge.	—
Conversations	Store AI chats.	—
Audit Logs	Track every important action.	—

12. Cache Layer

Cache is not required for correctness. It exists only for performance.

Possible Cache Items

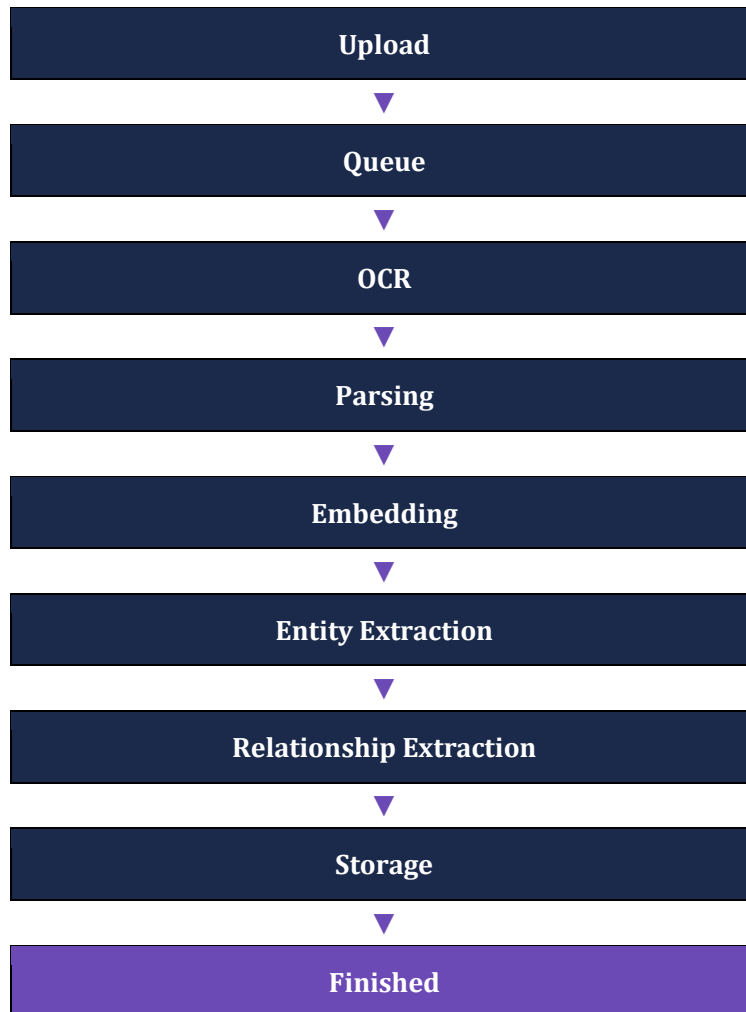
- User Sessions
- Recent Searches
- Frequently Used Documents
- Frequently Used Embeddings
- LLM Responses
- Permissions
- Organization Settings

Rule

Nothing critical should exist only inside cache. Everything important lives in permanent storage.

13. Background Processing

Uploading a document should return immediately. Processing happens later.



The user does not wait for this.

14. API Layer

The backend exposes several groups of APIs. Each module owns its own endpoints.

API Group	API Group
Authentication	AI Chat
User	Analytics
Organization	Notifications
Documents	Administration
Search	Knowledge

15. User Platform

The frontend is simply the interface. It owns no business logic.

Responsibilities

- Authentication
- Dashboard
- Upload
- Search
- Document Viewer
- AI Chat
- Knowledge Explorer
- Settings
- Analytics

Rule

The frontend never communicates directly with databases. Everything goes through APIs.

16. Event System

The platform generates events.

Examples

- Document Uploaded
- Document Processed
- Knowledge Updated
- Embedding Generated
- User Asked Question
- AI Generated Answer
- Document Deleted

Events allow different systems to react without tight coupling.

17. Security Requirements

- Authentication
- Authorization
- Encrypted storage
- Encrypted communication
- Audit logs
- Role-based permissions
- Document-level permissions
- Organization isolation (multi-tenancy)

18. Observability

Every important operation should be measurable.

Track

- Upload success rate
- Processing time
- OCR failures
- Embedding failures
- Search latency
- LLM latency
- AI answer quality
- API latency
- Error rate

19. Component Dependency Flow

One thing worth noticing: every system has a single responsibility. The dependencies should flow in one direction.

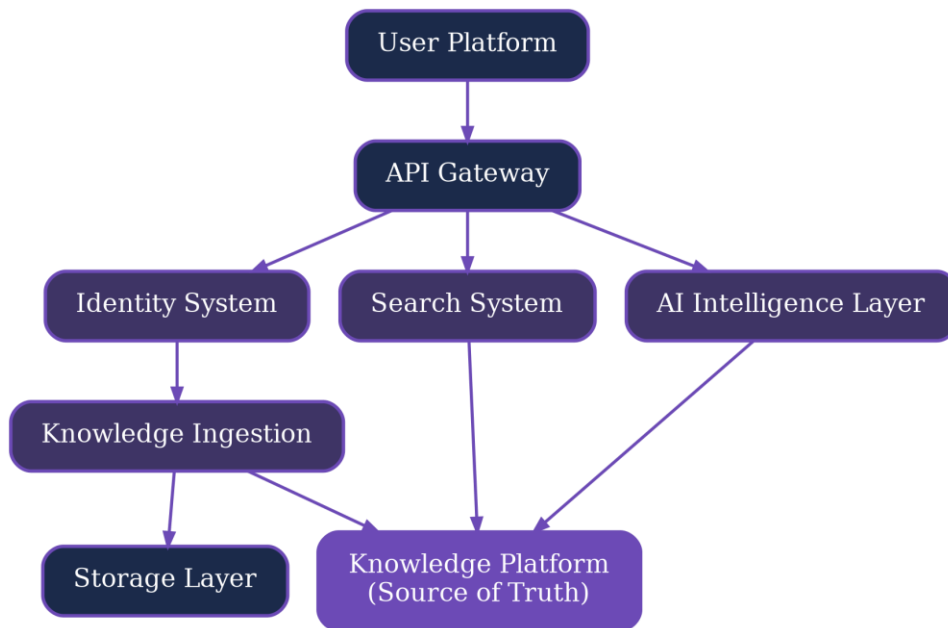


Figure — Platform component dependency flow

Rules

- The AI layer never owns data.
- The Search System is the only way to retrieve knowledge.
- The Knowledge Platform is the source of truth.
- The Ingestion System is the only component allowed to create or update knowledge.
- The User Platform only talks to APIs.

—. My Biggest Recommendation Before Writing Any Code

If I were leading this project, I would freeze this TRD first. Once the team agrees on:

- the six core systems,
- the responsibilities of each,
- the document lifecycle,
- the ownership boundaries,

then we don't touch the design for a few days.

Only after that would I create the implementation roadmap and map each responsibility to concrete technologies (e.g., Next.js, FastAPI, Qdrant, Neo4j, Redis, BullMQ, Gemini, etc.). That way, every technology choice is justified by a requirement rather than picked because it's popular.

Why this matters

This approach keeps a small team aligned and prevents major refactoring halfway through a 10–15 day sprint.